Claude Multi-Account Fine Tuning

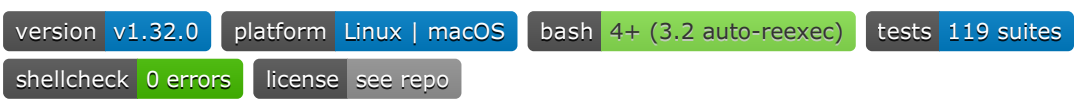
Claude Toolkit
Run many Claude Code accounts on one host — with one unified brain.
Why
- Features
- Quick start
- Daily commands
- Documentation
- Constitution hub engine
- Decision models (Jev) and the decide gateway
- Requirements
- Layout after unification
- How cross-account sync works
- Testing
- Releasing — the LIVE gate is mandatory

↩ Rollback
License

Claude Toolkit

Run many Claude Code accounts on one host — with one unified brain.

One alias per account. Shared history, memory, plugins, settings, and sessions across all of them. Plus turn any LLM API key into a Claude Code alias, share your whole plugin ecosystem with OpenCode, and auto-name a per-project session on every launch.



Quick start · Features · Commands · Guides · How it works

Why

Running more than one Claude Code account on a machine normally means siloed worlds: each account has its own conversation history, project memory, todos, plans, and installed plugins, and nothing carries over. Claude Toolkit collapses that into **one shared store** — every account is mostly symlinks into it — so switching accounts is just a shell alias, and your work, memory, and plugins follow you. Only the things that *must* stay private (credentials, auth state) remain per-account.

```
claude1     # account 1 — your shared projects, memory, plugins,
        sessions
claude2     # account 2 — the SAME shared projects, memory,
        plugins, sessions
deepseek    # the same ecosystem, running on DeepSeek's best model
kimi1       # Kimi Code account 1 — the same shared store, Kimi's
        OAuth subscription
kimi-deepseek  # Kimi Code on the SAME deepseek backend the no-
    prefix alias uses
```

Features

- **Unified multi-account store.** One alias per account (`claude1`, `claude2`, … or custom names). A single `~/.claude-shared/` holds `projects/`, `todos/`, `tasks/`, `plans/`, `history.jsonl`, `settings.json`, `plugins/`, `CLAUDE.md` and more; each account dir is symlinks into it.
- **Cross-account session resume.** The `projects`/session index in `.claude.json` is deep-merged across accounts on every launch — a session started under `claude1` shows up in `claude2`'s `--resume` list, and vice versa. Auth keys never cross.
- **Per-project auto-session naming.** Every bare alias launch resumes — or first-time creates — **one long-lived session per project root**, named after the directory in `kebab-case` (`Android 15` → `android-15`). Open `claude2` in a project and you're back in the same ongoing work, properly named — even if the session was previously unnamed. *(Color: a deterministic per-alias `/color` hint is printed; Claude Code's `/color` is TUI-only and can't be auto-applied — see SESSION_COLOR.md.)*
- **Any LLM as a Claude Code alias.** `claude-providers` turns every API key in your keys file into its own alias pointed at that provider's strongest model — fully dynamic (no hardcoded providers/URLs/models), secrets never leave the keys file. Provider guide →
- **Kimi Code CLI support.** `kimi1`, `kimi2`, … accounts with their own OAuth subscriptions, unified like Claude's; plus a `kimi-<id>` alias for every provider backend that opens **Kimi Code** on the same key. Kimi guide →

- **Share your ecosystem with OpenCode.** `claude-opencode-sync` exposes every Claude plugin's Skills + MCP servers + `CLAUDE.md` to a host-installed OpenCode in one command. OpenCode guide →
- **TOON encoding utility.** `toon.mjs` / `toon_encode.py` encode JSON to TOON (~40% fewer tokens for structured prompt data). TOON guide →
- **Jev decision layer.** A shadow-first, fail-open permission gate backed by TypeSafe's Jev decision models (local llmctl gateway, LAN pins, OpenRouter, or TypeSafe hosted) — never a chat alias, always a lightweight allow/ask/deny signal wired into every agent launch. Jev guide →
- **Constitution hub engine.** claude_toolkit is a *hub engine* of the Helix constitution: consumed **by reference** at `constitution/submodules/claude_toolkit`, depth 1, **never initialised recursively**, and it resolves its own dependencies (constitution, containers, challenges, LLMsVerifier, claude-code-router) with `scripts/resolve_deps.sh` instead of nesting them (Constitution §11.4.28(C) / §11.4.31). Hub-engine section →
- **llmctl validation gate + Vision QA.** Local-model aliases run through llmctl's known-answer validation gate (Constitution §11.4.277: `LLMCTL_VALIDATE_STRICT=1`, `llmctl require-validated <profile>`), the gate Vision QA (§11.4.278) depends on. llmctl guide →
- **Safe & reversible.** Every destructive step is backed up; `claude-rollback` restores everything. Idempotent installers. Verified on Linux and macOS.

Quick start

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/claude-
        toolkit/main/scripts/curl-install.sh | bash
```

Clones to `~/claude-toolkit` (or pulls if present), installs missing dependencies via your system package manager, runs the full setup, and wires the aliases into your shell. Re-run anytime to update.

Then open a new shell and:

```
claude-list-accounts     # see what's wired up
claude1                  # launch account 1 (auto-resumes this
        project's session)
```

Optional — the Jev decision layer (permission gating via System-One decision models, local-first via the llmctl `:8095` gateway):

```
claude-providers sync    # detect Jev endpoints into the ranked
        cache
jev-setup                # install gate/hooks/MCP per agent
        (shadow mode default)
```

```
claude1                    # "Jev decision layer: ACTIVE (…)" MOTD
        on launch
jev-stats                  # decisions, calibration, ESTIMATED
        tokens saved
```

Full walkthrough: docs/jev/Jev_Quickstart.md.

Manual install

A. Host already has ≥1 `~/.claude-*` account:

```
git clone https://github.com/vasic-digital/claude-toolkit.git
        claude-toolkit
cd claude-toolkit
bash scripts/install.sh
        # symlink scripts onto PATH, write managed alias
                                # file, source from rc files,
        unify existing
                                # accounts, install the TOON dep,
        refresh docs.
exec $SHELL -l
claude-list-accounts
```

B. Clean-slate host (Claude Code installed, no accounts yet):

```
git clone https://github.com/vasic-digital/claude-toolkit.git
        claude-toolkit
cd claude-toolkit
bash scripts/claude-bootstrap.sh --count 2 --yes     # provision
        claude1, claude2
exec $SHELL -l
claude1 /login && claude2 /login                      #
        authenticate each once
```

`--aliases personal,work` for custom names; `--dir-of NAME=PATH` to override a config dir. Both installers are **idempotent**.

Daily commands

Command	Purpose
`claude-list-accounts`	Tabular status: alias, config dir, creds, link health.
`claude-add-account`	Add an account. Interactive, or `--alias NAME --dir PATH --yes`.
`claude-remove-account --alias NAME`	Drop an alias; archive (default) or `--delete` its dir.

Command	Purpose
`claude-unify`	Re-merge state into the shared store (auto-detects `~/.claude-*`).
`claude-sync-state`	Fast `jq` sync of the `.claude.json` session index across accounts. Runs automatically around every launch.
`claude-session`	Per-project session helper (name/id/color/flags) used by the alias wrappers. See SESSION_COLOR.md.
`claude-bootstrap`	Clean-slate provisioning on a fresh host.
`claude-providers`	Create/refresh aliases for other LLM providers from your keys file.
`claude-providers quota` / `limits`	Report usage/limit windows for every native account and provider alias. See Provider_Aliases_User_Guide.md#14-quota-and-limits-reporting-quota--limits.
`kimi-add-account`	Add a Kimi Code account (`~/.kimi-code-<name>`); `--login` drives the interactive device flow.
`kimi-list-accounts`	Tabular status of detected Kimi accounts.
`kimi-unify`	Re-merge Kimi state into `$SHARED_DIR/kimi/`.
`kimi-remove-account`	Drop a Kimi account; archive (default) or `--delete` its dir.
`kimi-rollback`	Restore `.preunify.*` backups for the Kimi family.
`kimi-providers`	Kimi framing over `claude-providers` (`sync|list|show|verify|migrate-names`).
`kimi-providers quota` / `limits`	Same usage/limit report, reached via `kimi-providers`' catch-all forward. See Provider_Aliases_User_Guide.md#14-quota-and-limits-reporting-quota--limits.
`claude-usage report`	Token/cost usage report built on the real `ccusage` binary (day/model/session/project/provider breakdowns; default last 4 weeks; `json`/`md`/`csv`, written under `$SHARED_DIR/usage-reports/` by default). Native accounts are pooled (cannot be told apart) and "provider" is a best-effort catalog cross-reference — see `claude-usage --help` for the exact honesty contract. Auto-refreshed after every `claude-providers sync`/`sync --multi` (`CMA_SYNC_SKIP_USAGE_REFRESH=1` to opt out), not just on demand.
`claude-usage merge`	Combines `claude-usage report` JSON files from one or more source paths (`--from PATH[,PATH...]`, each a directory of `*.json` reports or a single file) into

Command	Purpose
	one consolidated report, re-summing any day/model/project/provider rows that key together. By default every project path is stripped to the placeholder `"coding agents"` (rows that collide after stripping are summed into one, not just relabeled) — `--anonymize-as NAME` picks a different label, `--real-names` opts out and keeps real paths distinct.
`claude-opencode-sync`	Expose Claude plugin Skills + MCP + `CLAUDE.md` to OpenCode.
`claude-export-docs`	Regenerate the long-form guide `.html`/`.pdf` from markdown.
`claude-rollback`	Restore `.preunify.*` backups and move the shared store aside.
`claude-providers sync-jev`	Jev-only fast path: re-probe decision endpoints and refresh the ranked `jev-endpoints.json` cache.
`jev-setup`	Install/audit (`--check`) the Jev decision layer per agent (hooks, MCP, plugin); edits agent homes only with consent (`--yes` / `JEV_SETUP_CONSENT=1`, backups first); `--mode shadow|enforce`, `--uninstall`.
`jev-stats`	Decision-layer report from `decisions.jsonl`: verdicts, latency, shadow calibration, ESTIMATED tokens saved. `--days N`, `--json`.
`jev-spec-setup <repo>`	Opt-in: install jev-spec CI templates (config + PR-verify workflow) into a target repo.

Documentation

Guide	What it covers
User Manual (start here)	One index over every guide, FAQ, diagram and verification doc below, with the hub-engine, validation-gate and decision-model entry points.
Provider Aliases User Guide	Turning LLM keys into aliases; transports, overrides, verification.
llmctl Integration Quickstart → User Guide → FAQ	Auto-recognizing local llmctl models as aliases; on-demand switching; honest context/LAN warnings; flow diagrams.
Kimi Accounts User Guide	Kimi Code accounts, `kimi-<id>` aliases, the one-time `kimi-*` → `kc-*` rename.
Jev Quickstart → User Guide → FAQ	The Jev decision layer: endpoint detection, the shadow-first permission

Guide	What it covers
	gate, MCP server, per-agent installer, stats/calibration; architecture diagram.
OpenCode Integration	Sharing Skills + MCP + `CLAUDE.md` with OpenCode.
Session & Color	Per-project auto-session naming + the per-alias color hint reality.
TOON Integration	Token-efficient JSON encoding utility.
Fine-Tuning deep dive	Full architecture walkthrough (`.html`/`.pdf` siblings).
CHANGELOG	Release history.

Constitution hub engine

claude_toolkit is one of the *hub engines* of the Helix constitution (`constitution/submodules/{llmctl,claude_toolkit,open_design}`):

- **Consumed by reference.** A project gets it at `constitution/submodules/claude_toolkit` (pinned by sha in the hub's `helix-deps.yaml`); it is never copied, never shipped inside a product artifact.
- **Never initialised recursively.** `git submodule update --init` stops at the engine; the only submodules it carries are the third-party `submodules/go` and `submodules/ccusage`.
- **No nested own-org copies.** Its dependencies are found by `scripts/resolve_deps.sh`: `CT_<NAME>_DIR` override → parent hub → `<root>/submodules/<name>` → sibling (Constitution §11.4.28(C), gate `CM-HUB-ENGINE-NO-OWNORG-NESTING`); `helix-deps.yaml` declares them (§11.4.31). Offline checks: `scripts/tests/test_helix_deps_manifest.sh`, `scripts/tests/test_gitmodules_no_ownorg.sh`.
- **Model governance it participates in.** §11.4.277 (llmctl model validation gate; STRICT mode, `llmctl require-validated`), §11.4.278 (HelixQA Vision UI QA, which reads screens through a gate-validated Vision model) and §11.4.279 (the decision-models layer: typed, non-chat answers that never become aliases — the Jev layer here). The toolkit forwards the gate's environment and propagates its refusals; it does not re-implement the gate (`scripts/tests/test_llmctl_vision_gate_wiring.sh`).

Diagram: hub-engine placement.

Decision models (Jev) and the decide gateway

Decision models answer typed questions (choice / noul / score) with probabilities; they have no chat endpoint, so they are **never aliases** (`classification: decision`, `alias: null`). The local llmctl *decide gateway* is **HTTPS with a mandatory bearer key** (`Authorization: Bearer $LLMCTL_API_KEY`, local CA at `$LLMCTL_HOME/cert/ca/ca.crt`); `/healthz` and `/readyz` are the only unauthenticated routes. The detector, gate and MCP server speak that (TLS with the llmctl CA, key from the env var named in the pin, never printed and never on a command line; plain http is opt-in via `CMA_JEV_ALLOW_HTTP=1`, loopback-only, unauthenticated, never for other hosts). Verified against a hermetic TLS stand-in, not a real gateway. See the Jev guide and the decide-gateway flow.

Requirements

`bash`, `rsync`, `jq`, `awk`. Optional: `node`+`npm` (TOON utility), `pandoc` (+ `weasyprint`/`wkhtmltopdf`/headless `chromium`) for doc regeneration.

```
# macOS (Homebrew)
brew install bash jq rsync gawk pandoc weasyprint
```

macOS ships bash 3.2; the scripts auto-re-exec under Homebrew bash when needed, and only touch `~/.zshrc` on Darwin.

Own-org dependencies are flat, not nested. This repo tracks no `vasic-digital/*` or `HelixDevelopment/*` submodule (the only submodules are the third-party `submodules/go` and `submodules/ccusage`). The Go router (`claude-code-router`), the optional `LLMsVerifier`, and `constitution`/`containers`/`challenges`/`superspec` are found by `scripts/resolve_deps.sh <name>`: `CT_<NAME>_DIR` override → parent hub (`<hub>/submodules/<name>`) → `<root>/submodules/<name>` → sibling `../<name>`, with a loud error and the clone command when nothing resolves. A standalone `curl-install.sh` run (no parent hub) clones the router over https into `$(dirname INSTALL_DIR)/claude_code_router` for you. The own-org dependency manifest is `helix-deps.yaml` (§11.4.31; validated by `scripts/tests/test_helix_deps_manifest.sh`).

Layout after unification

```
~/.claude-shared/                    # single source of truth
  projects/ todos/ tasks/ plans/ plugins/ sessions/ …
  CLAUDE.md  history.jsonl  settings.json  stats-cache.json

~/.claude-<account>/                 # per-account — mostly
```

```
symlinks
  .credentials.json                # PRIVATE — account-locked
  .claude.json                     # per-account state (non-auth
contents sync)
  mcp-needs-auth-cache.json        # PRIVATE
  projects -> ~/.claude-shared/projects   # every shared item is a
symlink

~/.claude-shared/kimi/             # Kimi shared store (mirrors
the above)
  AGENTS.md  plugins/  skills/  sessions/  session_index.jsonl

~/.kimi-code-<account>/            # per Kimi account — mostly
symlinks
  config.toml  credentials/  oauth/  device_id    # PRIVATE — per-
account
  sessions -> ~/.claude-shared/kimi/sessions      # shared items
are symlinks
```



```
~/.local/share/claude-multi-account/aliases.sh   # managed alias
file
```

How cross-account sync works

Claude Code keeps a `projects.<path>` map inside each account's `.claude.json` (`lastSessionId`, MCP status, project memory pointers). Without intervention, account A's projects are invisible to account B even though the JSONL transcripts live in the shared `projects/`. The toolkit fixes this in two layers:

1. **At unify time** — `claude-unify` deep-merges every account's `.claude.json` (rightmost-wins on scalars, recursive union of the `projects` subtree), writing each account's auth-private keys back untouched.
2. **At runtime** — the `cma_run` wrapper calls `claude-sync-state pull` before each launch and `push` after exit (a fast `jq` merge), so sessions created in one account appear in the others on the next launch.

Auth keys (`userID`, `oauthAccount`, `firstStartTime`, `claudeCodeFirstTokenDate`) never cross accounts.

Testing

```
bash scripts/tests/run-all.sh                # all hermetic suites (sandboxed $HOME)
bash scripts/tests/run-all.sh lib unify session   # subset by suffix
bash scripts/tests/run-proof.sh              # hermetic + live verifiers + evidence bundle
```

Tests use a sandboxed `$HOME` via `mktemp` — your real `~/.claude*` is never touched. The suite is **119 test files** (`ls scripts/tests/test_*.sh | wc -l`); a hermetic run on a host without a `claude-code-router` checkout fails the router-backed files (see CHANGELOG v1.32.0), and `shellcheck -S error` is clean (62 warning-level and 203 style-level suggestions remain); live verifiers (`verify_*_live.sh`) prove behavior against the real OpenCode/provider state and write inspectable evidence to `scripts/tests/proof/`.

Releasing — the LIVE gate is mandatory

```
claude-release-gate                          # sandbox suite + LIVE alias smoke
claude-release-gate --verify-providers       # + full LLMsVerifier model scan
claude-release-gate --provider poe           # smoke through a different provider
```

No release commit without a green `claude-release-gate`. The sandbox suite proves wrapper *logic* but is structurally blind to real-host state — v1.25.1 shipped fully green while every router alias on the real host was bricked by a PATH-shadowing `ccr` doppelgänger, and the live smoke then caught two more real defects (a mis-sliced HelixLLM context and ~330k tokens of auto-resumed session history) within minutes of existing. The gate drives the REAL generated alias through the REAL PATH → ccr → route-apply → proxy → backend and asserts the served reply plus the sink-side route. Fail-closed: any layer red = do not release.

↩ Rollback

```
claude-rollback        # == claude-unify --rollback
```

Restores every `.preunify.<timestamp>` backup and moves the shared store aside to `~/.claude-shared.removed.<timestamp>`.

License

See repository.